

Guided Lab — Intercept, Replay & Scrape a Storefront API

Day 2 · scripting track · ~2h · Burp Suite + Python, from a live browser session to a product table

Many sites look like static pages but are really a thin shell over a **private JSON API**. The browser holds a session (a bearer token, some cookies) and calls endpoints like `/search/productSearch` behind the scenes. In this lab you will: watch a real session in **Burp Suite**, find the endpoint that returns products and stock, read the exact request the browser sends, then **replay and paginate it in Python** to build an efficient product table.

The concrete target is a public e-commerce storefront built on Salesforce Commerce Cloud (the pattern behind `legami.com` and thousands of others). The method is general: the last section shows how the *same* steps apply to an industrial API you might discover through Shodan.

Rules of engagement. Only intercept traffic in **your own browser**, on a site you are authorised to test. You capture **your own** guest session — never paste someone else's token or cookies. Read the site's Terms of Service, keep request rates low, and collect only public catalogue data (never personal data or checkout/payment flows).

Part A — Set up Burp Suite (20 min)

Burp Suite is a local HTTP proxy: your browser sends traffic *through* Burp, so you can read and modify every request. Community Edition is free.

A1. Install and launch

```
# macOS
brew install --cask burp-suite
# Debian / the osint-lab container
sudo apt install burpsuite      # or download from
https://portswigger.net/burp/communitydownload
```

Launch Burp → **Temporary project** → **Use Burp defaults** → **Start Burp**. The proxy listens on `127.0.0.1:8080`.

A2. Route your browser through Burp

The clean way is a **dedicated browser profile** (so you never mix this with normal browsing), pointed at the proxy:

- Burp ships its own pre-configured browser: **Proxy** → **Intercept** → **Open Browser**. Use it — it trusts Burp's certificate automatically. *(If you prefer your own browser, set its HTTP/HTTPS proxy to `127.0.0.1:8080` and install Burp's CA from `http://burp/cert`, otherwise HTTPS pages will show certificate errors.)*

A3. Turn interception OFF, keep history ON

Proxy → **Intercept** → **Intercept is off**. Counter-intuitive, but we do not want to click "Forward" on every request. Instead we browse normally and read the passive log in **Proxy** → **HTTP history**.

Checkpoint: browse to any site in the Burp browser; requests appear in HTTP history. You are now watching the session.

Part B — Find the endpoint (25 min)

B1. Reproduce the action you want to automate

In the Burp browser, open the storefront and go to a **category listing** (e.g. stationery). Scroll, click "next page". Every product-loading call is now in HTTP history.

B2. Filter the noise

HTTP history is full of images, fonts and trackers. Filter it:

- Set the **Filter** bar to show only **parametrised requests** and hide images/CSS/scripts.
- Sort by **MIME type = JSON**, or type a keyword in the search box: `product`, `search`, `stock`.

B2. Predict. Among dozens of requests, which one is the product feed? What are the two tell-tale signs?

Answer. It is a request whose **response is JSON containing product objects** (name, price, an availability/stock field), and whose **URL path names the resource** — for a Commerce Cloud storefront it looks like: `GET /.../search/productSearch?c_sz=48&c_start=0&c_showAvailable=false&c_cgid=<category-id>` `c_sz` is the page size, `c_start` the offset, `c_cgid` the category. That is your endpoint.

B3. Read the request in full

Right-click the request → **Copy to file**, or read the **Request** panel. Note the three things Python will need:

1. **Method + URL + query parameters** (the pagination knobs: `c_start`, `c_sz`).
2. The **Authorization: Bearer eyJ...** header — the storefront's guest access token.
3. The **Cookie:** header — the session cookies (`dwsid`, `dwanonymous...`).

Where does the Bearer token come from? On Commerce Cloud it is a **SLAS guest token**: when the site first loads, the front-end silently calls a `/shopper/auth/.../token` endpoint and gets a short-lived token *for an anonymous shopper*. You do not need an account. You will capture yours the same way the browser did. **Never reuse a token you did not generate — it is tied to a session and expires.**

Part C — Replay one request in Python (25 min)

Now leave Burp and reproduce that single request in code. Right-click the request in Burp → **Copy as curl command** to get an exact template, then translate it to Python `requests`.

Install the tooling:

```
pip install requests pandas
```

C1. Predict. What is the minimum Python to send the captured GET with its headers and print the JSON keys?

Answer ```python import requests

```
BASE = "https://.../search/productSearch" # from Burp
HEADERS = { "User-Agent": "Mozilla/5.0",
"Accept": "application/json", "Authorization": "Bearer ", }
params = {"c_cgid": "", "c_sz": 48, "c_start": 0, "c_showAvailable": "false"}
```

```
r = requests.get(BASE, headers=HEADERS, params=params, timeout=20)
print(r.status_code)
print(list(r.json().keys())) ```
```

C2. Run it. Interpret the status code:

Code	Meaning	What to do
200	success	inspect <code>r.json()</code> ; find the products array and the total count
401	token missing/expired	re-capture a fresh token in Burp (Part B)
403	blocked / wrong headers	copy more headers from Burp (Referer, Origin)

Code	Meaning	What to do
429	rate-limited	slow down, honour <code>Retry-After</code>

C3. Print the fields you care about for the first product, so you know the JSON path:

```
first = r.json()["hits"][0]          # key name may differ; explore r.json() first
print(first["productName"], first["price"], first.get("orderable"))
```

Part D — Paginate and build the table (25 min)

One page is proof of concept. Now walk every page and load the results into a `pandas` table.

D1. Predict. The API returns a page of `c_sz` items starting at `c_start`, plus a `total`. How do you loop until you have them all?

Answer. Start at `c_start = 0`, read `total` from the first response, then increase `c_start` by the page size until it reaches `total`.

D2. Here is the complete, robust scraper. It uses a **session** (connection reuse), an **exponential backoff** on 429, a **human cadence** between calls, and writes a CSV. The values in `<...>` come from your Burp capture.

```
#!/usr/bin/env python3
"""scrape_products.py - replay a captured storefront search endpoint and paginate it."""
import time, requests, pandas as pd

BASE = "https://<host>/.../search/productSearch"      # from Burp
TOKEN = "<PASTE-YOUR-CAPTURED-TOKEN>"                # your own SLAS guest token
CGID = "<category-id>"                                # from Burp (c_cgid)
PAGE = 48

def make_session():
    s = requests.Session()
    s.headers.update({
        "User-Agent": "Mozilla/5.0",
        "Accept": "application/json",
        "Authorization": f"Bearer {TOKEN}",
    })
    return s

def get_page(s, start):
    params = {"c_cgid": CGID, "c_sz": PAGE, "c_start": start, "c_showAvailable":
```

```

"false"}
    for attempt in range(4):
        # backoff on rate limit
        r = s.get(BASE, params=params, timeout=20)
        if r.status_code == 429:
            wait = int(r.headers.get("Retry-After", 2 ** attempt))
            print(f" 429 -> wait {wait}s"); time.sleep(wait); continue
        r.raise_for_status()
        return r.json()
    raise RuntimeError("too many retries")

def main():
    s = make_session()
    first = get_page(s, 0)
    total = first["total"]
    hits = list(first["hits"])
    print(f"total products: {total}")
    start = PAGE
    while start < total:
        hits += get_page(s, start)["hits"]
        start += PAGE
        time.sleep(0.3)
    # keep only useful columns; adapt the field names to the real JSON
    df = pd.json_normalize(hits)[["productId", "productName", "price", "orderable"]]
    df.to_csv("products.csv", index=False)
    print(f"rows: {len(df)}")
    print(df.head(5).to_string(index=False))
    print("in stock:", (df["orderable"] == True).sum(), "/", len(df))

if __name__ == "__main__":
    main()

```

D3. Run it and read the output:

```

python3 scrape_products.py
column -s, -t products.csv | head

```

You now have every product of the category, with stock, in a CSV — built entirely from one intercepted request.

If you have no live storefront to hand, the exact same code shape works against the public practice API <https://dummyjson.com/products> (use params `limit/skip` instead of `c_sz/c_start`, and keys `products/total`). Prove your loop and table on it first, then swap in the captured endpoint.

Part E — Make it resilient (15 min)

Answer and implement, using your own script.

E.1 The token expires after a while and you start getting `401` mid-crawl. Add a check: on `401`, stop with a clear message telling the operator to re-capture a token. (Bonus: script the guest-token call itself so it refreshes automatically.)

E.2 You want to be a good citizen. Add a `--delay` argument and a hard cap on pages, and log one line per page fetched.

E.3 The `stock/orderable` field is sometimes missing. Make `pandas` handle missing values instead of crashing (`errors`, `.get`, `dropna`).

E.4 De-duplicate: a product can appear in two categories. Key the table on `productId` and drop duplicates.

Part F — Why this matters: from a storefront to a Shodan-found API (10 min)

The technique you just practised is exactly how an analyst approaches an **undocumented API discovered in the wild**. Suppose Shodan surfaces a host exposing a JSON service tied to a manufacturer's logistics or parts catalogue (a Boeing or COMAC supplier portal, say):

1. **Discovery** — Shodan/Censys reveals the host, open ports and often the framework (which tells you the likely API shape).
2. **Observe** — drive the real front-end through Burp, exactly as here, to learn the auth scheme and the endpoints.
3. **Understand the contract** — method, params, pagination, the token's origin and lifetime.
4. **Replay** — reproduce one authorised request in Python.
5. **Scale carefully** — paginate with backoff and a human cadence, store with provenance (URL, UTC, hash — the Day 1 discipline).

The method does not change; only the target and the legal frame do. Against anything you do not own, you need explicit authorisation (a scope, an engagement letter). The skill is neutral; the authorisation is what makes it legitimate.

Deliverable. A short note: the endpoint you found, an annotated screenshot of the Burp request (token and cookies **redacted**), your working `scrape_products.py`, and the first ten rows of `products.csv`. State which site you tested and under what authorisation.